

Laboratorio práctico de Docker — Parte I: **Dockerfile**

Obxectivo xeral: Construír imaxes reproducibles paso a paso, introducindo progresivamente instalación de paquetes, variables de contorno, configuracións, boas prácticas de seguridade e optimizacións.

Requisitos previos

- Docker instalado.
- Editor de texto.
- Sistema Linux/WSL recomendado.
- Crear unha carpeta raíz `docker-lab/` e entrar nela.

Estrutura suxerida para esta parte:

```
docker-lab/
├── 01-cowsay/
├── 02-java-tar/
├── 03-configs-nginx/
├── 04-python-app/
├── 05-nonroot/
├── 06-multistage/
├── 07-health-entrypoint/
├── 08-optimizations/
└── 09-secrets-buildkit/
```

En cada subcarpeta inclúe un `Dockerfile`, os ficheiros auxiliares necesarios e un `README.md` curto con comandos de proba.

1) Instalar un paquete simple (cowsay)

Que vas facer: crear unha imaxe base sinxela, instalar un paquete do sistema e definir un comportamento por defecto do contedor.

Pasos detallados:

1. Base `debian:12-slim` (lixeira e estable).
2. `apt-get update` && `apt-get install cowsay` sen recomendados, e limpeza de caché.
3. `ENTRYPOINT` a `/usr/games/cowsay` e `CMD` cun texto por defecto.

Ficheiros (`01-cowsay/Dockerfile`):

```
FROM debian:12-slim
RUN apt-get update \
    && apt-get install -y --no-install-recommends cowsay \
    && rm -rf /var/lib/apt/lists/*
```

```
ENTRYPOINT ["/usr/games/cowsay"]  
CMD ["Hola, Docker!"]
```

Probas:

```
cd 01-cowsay  
docker build -t lab/cowsay:1 .  
docker run --rm lab/cowsay:1 "Boas desde un contedor"
```

Puntos clave: capas, diferenza **ENTRYPOINT** vs **CMD**, limpeza de capas.

Criterios de éxito: imprime o ASCII-art de vaca coa mensaxe indicada.

2) Instalar Java con **apt** e descargar unha app **.tar.gz** (Tomcat) con variables **ENV**

Que vas facer: montar unha imaxe de execución Java configurando variables de contorno e descargando Tomcat en **.tar.gz** durante a construción.

Pasos detallados:

1. Base **debian:12-slim**.
2. Instalar Java con **apt**: **openjdk-17-jre-headless**, e utilidades (**curl**, **ca-certificates**, **tar**).
3. Definir **ENV** en **dúas liñas** para empregar variables encadeadas.
4. Descargar Tomcat con **curl**, extraelo en **/opt** e normaliza o directorio.
5. **EXPOSE 8080** e **CMD ["catalina.sh", "run"]**.

Ficheiros (**02-java-tar/Dockerfile**):

```
FROM debian:12-slim  
  
ARG TOMCAT_VERSION=10.1.29  
  
# Defínese CATALINA_HOME e ampliase PATH en dúas liñas (expansión segura)  
ENV CATALINA_HOME=/opt/tomcat  
ENV  
PATH="$CATALINA_HOME/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"  
  
RUN apt-get update \  
  && apt-get install -y --no-install-recommends openjdk-17-jre-headless curl ca-  
certificates tar \  
  && rm -rf /var/lib/apt/lists/* \  
  && mkdir -p /opt \  
  && curl -fsSL -o /tmp/tomcat.tar.gz \  
    https://archive.apache.org/dist/tomcat/tomcat-  
10/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz \  
  && tar -xzf /tmp/tomcat.tar.gz -C /opt \  
  && mv /opt/tomcat /opt/tomcat-${TOMCAT_VERSION}
```

```
&& mv /opt/apache-tomcat-${TOMCAT_VERSION} "$CATALINA_HOME" \  
&& rm /tmp/tomcat.tar.gz  
  
EXPOSE 8080  
CMD ["catalina.sh", "run"]
```

Probas:

```
cd 02-java-tar  
docker build -t lab/tomcat:10 .  
docker run --rm -p 8080:8080 lab/tomcat:10  
# Abrir http://localhost:8080
```

Puntos clave: ARG vs ENV, reproducibilidade das descargas, layout en /opt.

Criterios de éxito: Tomcat atende en <http://localhost:8080>.

3) Copiar un ficheiro de configuración sinxelo: **Nginx**

Que vas facer: empregar Nginx e copiar un [nginx.conf](#) personalizado á imaxe xunto cunha páxina estática. Validar a configuración no *build* para evitar sorpresas en runtime.

Estrutura ([03-configs-nginx/](#)):

```
03-configs-nginx/  
├─ Dockerfile  
├─ nginx.conf  
└─ html/  
    └─ index.html
```

Ficheiros

[03-configs-nginx/Dockerfile](#):

```
FROM debian:12-slim  
  
# 1) Instalar Nginx e utilidades mínimas  
RUN apt-get update \  
&& apt-get install -y --no-install-recommends nginx ca-certificates \  
&& rm -rf /var/lib/apt/lists/*  
  
# 2) Copiar a configuración e o contido estático  
COPY nginx.conf /etc/nginx/nginx.conf  
COPY html/ /usr/share/nginx/html/  
  
# 3) Validar config en build (falla se hai erros)  
RUN nginx -t
```

```
# 4) Expor porto e executar Nginx en primeiro plano
EXPOSE 8080
CMD ["nginx","-g","daemon off;"]
```

03-configs-nginx/nginx.conf:

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    # Servidor principal
    server {
        listen 8080;
        server_name _;

        # Directorio coa páxina estática
        root /usr/share/nginx/html;
        index index.html;

        # Endpoint de comprobación
        location /health {
            return 200 'ok';
        }

        # Ruta por defecto
        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

03-configs-nginx/html/index.html:

```
<!doctype html>
<html>
  <body>
    <h1>Ola desde Nginx en Docker</h1>
    <p>Este sitio está servido cunha config personalizada.</p>
  </body>
</html>
```

Probas:

```
cd 03-configs-nginx
docker build -t lab/nginx:conf .
docker run --rm -p 8080:8080 lab/nginx:conf
# Abrir http://localhost:8080 e http://localhost:8080/health
```

Puntos clave: `COPY` de configs, validación con `nginx -t`, foreground con `daemon off;`.

Criterios de éxito: `/` e `/health` responden correctamente.

4) Construcción de aplicaciones Python con `requirements.txt` (Flask)

Que vas hacer: empaquetar unha API Flask simple demostrando o patrón de caché clásico (copiar `requirements.txt` antes do código).

Estrutura (`04-python-app/`):

```
04-python-app/
├── Dockerfile
├── requirements.txt
└── app.py
```

`requirements.txt`:

```
Flask==3.0.3
```

`app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def hello():
    return {"msg": "hola dende docker"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

`Dockerfile`:

```
FROM python:3.12-slim
WORKDIR /app
# Cache de deps
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copiar o código
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Probas:

```
cd 04-python-app
docker build -t lab/flask:1 .
docker run --rm -p 5000:5000 lab/flask:1
# GET http://localhost:5000 → {"msg":"hola dende docker"}
```

Puntos clave: capas e invalidación de caché, `--no-cache-dir`.

Criterios de éxito: responde JSON en `/`.

5) Usuario de aplicación (non executar como root)

Que vas facer: crear un usuario/grupo non privilexiado e garantir permisos correctos sobre os ficheiros.

Ficheiros (`05-nonroot/Dockerfile`):

```
FROM python:3.12-slim
WORKDIR /app

# Crear usuario/grupo do sistema
RUN groupadd -r app && useradd -r -g app app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt \
  && mkdir -p /app \
  && chown -R app:app /app

# Copiar como propietario app
COPY --chown=app:app app.py .

USER app
EXPOSE 5000
CMD ["python", "app.py"]
```

Probas (reuse da app do paso 4):

```
cd 05-nonroot
cp ../04-python-app/requirements.txt .
cp ../04-python-app/app.py .
docker build -t lab/flask:nonroot .
```

```
docker run --rm -p 5000:5000 lab/flask:nonroot
# Dentro do contedor: id → non root
```

Puntos clave: menor privilexio, impacto en volumes e montaxes, `COPY --chown`.

Criterios de éxito: app funciona e `id` mostra UID non-root.

6) Construción multietapa (multistage)

Que vas facer: separar construción e runtime para obter imaxes máis pequenas e seguras.

Ficheiros (`06-multistage/Dockerfile`):

```
# Etapa de construción
FROM python:3.12 as builder
WORKDIR /build
COPY requirements.txt .
RUN pip wheel --wheel-dir=/build/wheels -r requirements.txt
COPY app.py .

# Etapa final mínima
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /build/wheels /wheels
RUN pip install --no-cache-dir /wheels/*
COPY app.py .
# usuario non-root xenérico
USER 65532:65532
EXPOSE 5000
ENTRYPOINT ["python", "app.py"]
```

Probas:

```
cd 06-multistage
cp ../04-python-app/requirements.txt .
cp ../04-python-app/app.py .
docker build -t lab/flask:ms .
docker images | grep lab/flask
```

Puntos clave: reducir superficie de ataque, comparar tamaños de imaxe.

Criterios de éxito: imaxe final máis pequena mantendo funcionalidade.

7) HEALTHCHECK, ENTRYPOINT con tini e sinais

Que vas facer: engadir un `HEALTHCHECK` HTTP e usar `tini` para un PID 1 correcto (sinais, procesos zombie).

Estrutura (`07-health-entrypoint/`):

```
07-health-entrypoint/  
├─ Dockerfile  
├─ requirements.txt  
├─ app.py  
└─ entrypoint.sh
```

entrypoint.sh:

```
#!/bin/sh  
set -e  
exec "$@"
```

Dockerfile:

```
FROM python:3.12-slim  
RUN apt-get update && apt-get install -y --no-install-recommends tini \  
    && rm -rf /var/lib/apt/lists/*  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
COPY app.py .  
COPY entrypoint.sh /usr/local/bin/entrypoint.sh  
RUN chmod +x /usr/local/bin/entrypoint.sh  
  
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \  
    CMD python -c "import urllib.request;  
    urllib.request.urlopen('http://localhost:5000/').read()" || exit 1  
  
ENTRYPOINT ["/usr/bin/tini", "--", "/usr/local/bin/entrypoint.sh"]  
CMD ["python", "/app/app.py"]
```

Probas: construír e ver **STATUS** → **healthy**. Parada limpa con **Ctrl+C**.

Puntos clave: liveness/readiness (aquí probe simple), propagación de sinais.

Criterios de éxito: **docker ps** mostra **healthy** e parada limpa.

8) Optimizacións e boas prácticas

Que vas facer: aplicar boas prácticas de *image hygiene*: etiquetas OCI, **.dockerignore**, ENV útiles, instalación mínima e usuario fixo.

Estrutura (08-optimizations/):

```
08-optimizations/  
├─ Dockerfile
```

```
|— requirements.txt
|— app.py
|— .dockerignore
```

Dockerfile:

```
FROM python:3.12-slim
LABEL org.opencontainers.image.source="https://example.local/lab" \
      org.opencontainers.image.description="Exemplo optimizado" \
      org.opencontainers.image.licenses="MIT"
WORKDIR /app

# .dockerignore evita copiar lixo á build context
COPY .dockerignore /app/.dockerignore

ARG APP_VERSION=1.0.0
ENV APP_VERSION=$APP_VERSION PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

RUN apt-get update \
  && apt-get install -y --no-install-recommends curl \
  && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
RUN useradd -r -u 10001 app && chown -R app:app /app
USER 10001

CMD ["python", "app.py"]
```

.dockerignore:

```
__pycache__/
*.pyc
.git/
.vscode/
venv/
*.log
```

Puntos clave: metadatos OCI, impacto de `.dockerignore` na caché, reproducibilidade.

Criterios de éxito: imaxe máis pequena e con metadatos visibles.

9) Segredos en construción (BuildKit)

Que vas facer: usar BuildKit `--mount=type=secret` para ler credenciais en build sen deixalas na imaxe.

Estrutura (09-secrets-buildkit/):

```
09-secrets-buildkit/  
├── Dockerfile  
└── .npm_token    # exemplo de segredo (NON subir a repo)
```

Dockerfile:

```
# syntax=docker/dockerfile:1.7  
FROM alpine:3.20  
RUN --mount=type=secret,id=NPM_TOKEN \  
    echo "Usando token (non persistirá na capa)" >/dev/null  
CMD ["sh","-c","echo listo"]
```

Construción:

- Linux/MacOS:

```
cd 09-secrets-buildkit  
DOCKER_BUILDKIT=1 docker build --secret id=NPM_TOKEN,src=./.npm_token -t lab/with-secret .
```

- Windows:

```
cd 09-secrets-buildkit  
$env:DOCKER_BUILDKIT=1  
docker build --secret id=NPM_TOKEN,src=./.npm_token -t lab/with-secret .
```

Puntos clave: diferenciar build args/env/secrets; non quedan en capas nin en `docker history`.

Criterios de éxito: construción OK e segredo ausente do historial.

Cheatsheet e problemas habituais

Comandos útiles:

```
docker build -t nome:tag .  
docker images  
docker ps -a  
docker run --rm -p 8080:80 imaxe  
docker exec -it nome sh  
docker system df
```

Problemas comúns e solucións rápidas:

- Porto en uso → cambia `HOST:CONTAINER`.
- Caché stale → `--no-cache` ou tocar `requirements.txt`.
- Permisos en volumes → coidar `uid/gid` do usuario da imaxe.
- Dif. `ENV/ARG/secrets` → usar cada un para o seu propósito.